cadence

HPNFC Flash controller User Guide

Product Version 4.1.0

September 2017

© 1996-2017 Cadence Design Systems, Inc. All rights reserved.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522. All other trademarks are the property of their respective holders.

Restricted Permission: This document is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this document, or any portion of it, may result in civil and criminal penalties. Except as specified in this permission statement, this document may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This document contains the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only in accordance with, a written agreement between Cadence and its customer.

Unless otherwise agreed to by Cadence in writing, this statement grants Cadence customers permission to print one (1) hard copy of this document subject to the following conditions:

1. This document may not be modified in any way.

2. Any authorized copy of this document or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.

3. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

Disclaimer: INFORMATION IN THIS DOCUMENT IS SUBJECT TO CHANGE WITHOUT NOTICE AND DOES NOT REPRESENT A COMMITMENT ON THE PART OF CADENCE. EXCEPT AS MAY BE EXPLICITLY SET FORTH IN A WRITTEN AGREEMENT BETWEEN CADENCE AND ITS CUSTOMER, CADENCE DOES NOT MAKE, AND EXPRESSLY DISCLAIMS, ANY REPRESENTATIONS OR WARRANTIES AS TO THE COMPLETENESS, ACCURACY OR USEFULNESS OF THE INFORMATION CONTAINED IN THIS DOCUMENT. CADENCE DOES NOT WARRANT THAT USE OF SUCH INFORMATION WILL NOT INFRINGE ANY THIRD PARTY RIGHTS, AND CADENCE DISCLAIMS ALL IMPLIED WARRANTIES, INCLUDING MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. CADENCE DOES NOT ASSUME ANY LIABILITY FOR DAMAGES OR COSTS OF ANY KIND THAT MAY RESULT FROM USE OF SUCH INFORMATION. CADENCE CUSTOMER HAS COMPLETE CONTROL AND FINAL DECISION-MAKING AUTHORITY OVER ALL ASPECTS OF THE DEVELOPMENT, MANUFACTURE, SALE AND USE OF CUSTOMER'S PRODUCT, INCLUDING, BUT NOT LIMITED TO, ALL DECISIONS WITH REGARD TO DESIGN, PRODUCTION, TESTING, ASSEMBLY, QUALIFICATION, CERTIFICATION, INTEGRATION OF CADENCE PRODUCTS, INSTRUCTIONS FOR USE, LABELING AND DISTRIBUTION, AND CADENCE EXPRESSLY DISAVOWS ANY RESPONSIBILITY WITH REGARD TO ANY SUCH DECISIONS REGARDING CUSTOMER'S PRODUCT.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227- 14 and DFAR252.227-7013 et seq. or its successor.

HPNFC Flash controller User Guide: User Guide

Cadence Design Systems

Hardware Identifier:4984ddc02d80b6c91ef1f3653c8d43b6

Table of Contents

1. Acronyms 1
2. Document Purpose 2
3. Core Driver Description 3
    3.1. Features 3
    3.2. Software Stack Layering 3
4. Core Driver Usage Model 4
    4.1. Description 4
    4.2. Initialization 4
    4.3. NAND flash memory command exection 5
        4.3.1. Command DMA mode 5
        4.3.2. PIO mode 6
        4.3.3. Generic mode 6
        4.3.4. Operation result 7
    4.4. Switching timing work mode 7
    4.5. ECC configuration 7
    4.6. Device layout configuration 7
5. Configuration and Hardware Operation Information 9
    5.1. Introduction 9
6. Dynamic Data Structures 10
    6.1. Introduction 10
    6.2. HPNFC_ControllerInfo_s 10
    6.3. HPNFC_BchConfigInfo_s 11
    6.4. HPNFC_Config_s 11
    6.5. HPNFC_LunInterleavedConfig_s 11
    6.6. HPNFC_DeviceSubLayout_s 12
    6.7. HPNFC_DeviceLayout_s 12
    6.8. HPNFC_StatusPollingConfig_s 12
    6.9. HPNFC_AutoStatusConfig_s 13
    6.10. HPNFC_TransferConfig_s 13
    6.11. HPNFC_MultiplaneConfig_s 14
    6.12. HPNFC_CacheConfig_s 14
    6.13. HPNFC_EccConfiguration_s 14
    6.14. HPNFC_EraseDetectConfig_s 15
    6.15. HPNFC_PhyConfig_s 15
    6.16. HPNFC_GenReqInputAddress_s 16
    6.17. HPNFC_GenReqInputData_s 16
    6.18. HPNFC_GenReqInput_s 17
    6.19. HPNFC_OperationResult_s 17
    6.20. HPNFC_GenReq_s 18
    6.21. HPNFC_PioReq_s 18
    6.22. HPNFC_RstReq_s 19
    6.23. HPNFC_CdmaReq_s 19
    6.24. HPNFC_CdmaSyncConfig_s 19
    6.25. HPNFC_CdmaCmd_s 20
    6.26. HPNFC_SysReq_s 21
    6.27. HPNFC_WorkModeSpecificRequest_s 21
    6.28. HPNFC_CmdReq_s 22
    6.29. HPNFC_DebugReq_s 22
    6.30. HPNFC_DiConfig_s 22

6.31. HPNFC_DiErrInjConfig_s 23
6.32. HPNFC_DmaConfig_s 23
6.33. HPNFC_RemapRecord_s 24
6.34. HPNFC_PSlcConfig_s 24
6.35. HPNFC_ZqCalConfig_s 24
6.36. HPNFC_TimeoutConfig_s 25
6.37. HPNFC_DevInfo_s 25
6.38. HPNFC_TransferDirection 25
6.39. HPNFC_CdmaCmdType 26
6.40. HPNFC_PioCmdType 26
6.41. HPNFC_GenericCmdType 27
6.42. HPNFC_WorkMode 31
6.43. HPNFC_DmaMode 31
6.44. HPNFC_OperationStatus 31
6.45. HPNFC_ReqType 33
6.46. HPNFC_LunStatSelect 33
6.47. HPNFC_CdmaSyncStartCond 34
6.48. HPNFC_CdmaSyncEndOp 34
6.49. HPNFC_BootStatus 34
6.50. HPNFC_DebugSynchMode 35
6.51. HPNFC_DebugIfType 36
6.52. HPNFC_DiParityType 36
6.53. HPNFC_DiErrInjType 36
6.54. HPNFC_PlanesNumber 37
6.55. HPNFC_MplPrgSeq 37
6.56. HPNFC_MplRdSeq 38
6.57. HPNFC_MplEraseSeq 39
6.58. HPNFC_LastWrCmdSeq 39
6.59. HPNFC_CopyBckCmdSeq 39
6.60. HPNFC_TargetPerPackage 40
6.61. HPNFC_DevType 40
6.62. HPNFC_ControllerInfo 41
6.63. HPNFC_BchConfigInfo 41
6.64. HPNFC_Config 41
6.65. HPNFC_LunInterleavedConfig 41
6.66. HPNFC_DeviceSubLayout 41
6.67. HPNFC_DeviceLayout 42
6.68. HPNFC_StatusPollingConfig 42
6.69. HPNFC_AutoStatusConfig 42
6.70. HPNFC_TransferConfig 42
6.71. HPNFC_MultiplaneConfig 42
6.72. HPNFC_CacheConfig 43
6.73. HPNFC_EccConfiguration 43
6.74. HPNFC_EraseDetectConfig 43
6.75. HPNFC_PhyConfig 43
6.76. HPNFC_GenReqInputAddress 43
6.77. HPNFC_GenReqInputData 44
6.78. HPNFC_GenReqInput 44
6.79. HPNFC_OperationResult 44
6.80. HPNFC_GenReq 44
6.81. HPNFC_PioReq 44

6.82. HPNFC_RstReq 44
6.83. HPNFC_CdmaReq 45
6.84. HPNFC_CdmaSyncConfig 45
6.85. HPNFC_CdmaCmd 45
6.86. HPNFC_SysReq 45
6.87. HPNFC_WorkModeSpecificRequest 45
6.88. HPNFC_CmdReq 46
6.89. HPNFC_DebugReq 46
6.90. HPNFC_DiConfig 46
6.91. HPNFC_DiErrInjConfig 46
6.92. HPNFC_DmaConfig 46
6.93. HPNFC_RemapRecord 47
6.94. HPNFC_PSlcConfig 47
6.95. HPNFC_ZqCalConfig 47
6.96. HPNFC_TimeoutConfig 47
6.97. HPNFC_DevInfo 47
6.98. HPNFC_SlaveDmaHandler 47
7. Driver API Object 49
7.1. Introduction 49
7.2. HPNFC_Probe 49
7.3. HPNFC_Init 49
7.4. HPNFC_Isr 50
7.5. HPNFC_Start 50
7.6. HPNFC_Stop 51
7.7. HPNFC_Destroy 51
7.8. HPNFC_AssembleCdmaDesc 51
7.9. HPNFC_ExecCmdReq 52
7.10. HPNFC_SetWorkMode 53
7.11. HPNFC_SetEccConfig 53
7.12. HPNFC_GetEccConfig 54
7.13. HPNFC_SetTransferConfig 55
7.14. HPNFC_GetTransferConfig 55
7.15. HPNFC_CdmaGetFailedDesc 56
7.16. HPNFC_CdmaGetNextFailedDesc 56
7.17. HPNFC_CdmaDescGetOperationResult 57
7.18. HPNFC_ReqGetOperationResult 58
7.19. HPNFC_CdmaParseDesc 58
7.20. HPNFC_SetStatusPollingConfig 59
7.21. HPNFC_GetStatusPollingConfig 60
7.22. HPNFC_SetAutoStatusConfig 60
7.23. HPNFC_GetAutoStatusConfig 61
7.24. HPNFC_GetDeviceLayout 61
7.25. HPNFC_SetDeviceSubLayout 62
7.26. HPNFC_SetLunInterleavedConfig 62
7.27. HPNFC_GetLunInterleavedConfig 63
7.28. HPNFC_GetRnb 64
7.29. HPNFC_GetBootStatus 64
7.30. HPNFC_GetMultiplaneConfig 65
7.31. HPNFC_SetMultiplaneConfig 65
7.32. HPNFC_GetCacheConfig 66
7.33. HPNFC_SetCacheConfig 67

7.34. HPNFC_GetBchConfigInfo 67
7.35. HPNFC_GetControllerInfo 68
7.36. HPNFC_GetCdmaDescSize 68
7.37. HPNFC_GetDebugData 69
7.38. HPNFC_SetDebugIfType 69
7.39. HPNFC_GetDebugIfType 70
7.40. HPNFC_SetDiConfig 71
7.41. HPNFC_GetDiConfig 71
7.42. HPNFC_SetDiErrInjConfig 72
7.43. HPNFC_GetDiErrInjConfig 72
7.44. HPNFC_SetDmaConfig 73
7.45. HPNFC_GetDmaConfig 74
7.46. HPNFC_WriteRemapRecord 74
7.47. HPNFC_ReadRemapRecord 75
7.48. HPNFC_ClearRemapTable 76
7.49. HPNFC_GetRemapRecordCount 76
7.50. HPNFC_SetRemapMask 77
7.51. HPNFC_GetRemapMask 77
7.52. HPNFC_SetRemapEnable 78
7.53. HPNFC_GetRemapEnable 78
7.54. HPNFC_SetPSlcConfig 79
7.55. HPNFC_GetPSlcConfig 80
7.56. HPNFC_SetZqCalConfig 80
7.57. HPNFC_GetZqCalConfig 81
7.58. HPNFC_SetZqCalEnable 81
7.59. HPNFC_GetZqCalEnable 82
7.60. HPNFC_GetZqStat 83
7.61. HPNFC_ClearZqStat 83
7.62. HPNFC_SetTimeoutConfig 84
7.63. HPNFC_GetTimeoutConfig 84
7.64. HPNFC_SetCePinReduction 85
7.65. HPNFC_GetCePinReduction 85
7.66. HPNFC_GetDeviceInfo 86

List of Tables

4.1. Specific request depends on request type 5
5.1. Configuration and Hardware Operation Information 9

Chapter 1. Acronyms

API	Application Programming Interface.
Core Driver	Cadence Firmware component that provides IP programming abstraction.
CPS	Cadence Platform Services. A set of basic platform specific access functions acting as hardware abstraction layer for the core driver to operate.
AHB	Advanced High Performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ARM	Advanced RISC Machines
HRP	Hard Reference Platform
IP	Intellectual Property
ONFI	Open NAND Flash Interface

Chapter 2. Document Purpose

This document is intended to serve as an overview and a reference for customers looking to use the Cadence NAND Flash Firmware Core Driver for the Cadence NAND Flash Core.

Chapter 3. Core Driver Description

The FW provided by Cadence is of 2 types:

1. Production Code: Code built to production standards. i.e. it is fully verified with positive and negative testing.
2. Reference Code: This is example code for the platform specific portions of the customers code base. Reference code is only provided as an example to help the customer with his environment. Ideally code snippets from this will be looked at, and the customer will create his own versions of the reference code that is specific to his environment.

3.1. Features

The delivered software will interface with all major functions of the Controller. There are four types of communication with Flash memory.

1. Execution commands on NAND flash memory in CMDA mode.
2. Execution commands on NAND flash memory in PIO mode.
3. Execution commands on NAND flash memory in Generic mode.
4. Execution data transfer in Slave and Master DMA mode.
5. Configuration of ECC module
6. Working in interrupt and polling mode.
7. Configuration of PHY and timing registers
8. Configuration of status polling and auto status
9. Switching to higher modes NV_DDR, NV_DDR2, NV_DDR3 and toggle.
10. Configuration LUN interleaving
11. Configuration multiplane commands
12. Configuration cache commands

3.2. Software Stack Layering

The software stack for the HPNFC Controller Driver shall consist of the following elements:

- Register access block
- Core driver block
- Command DMA block
- PHY block

API to implementation blocks interface The register access block shall provide register access by name, depending on the controller configuration. The core driver block shall use the registed access block to communicate with the flash device, CDMA block to preparing commands and handling status parsing, PHY block to configure PHY module. The API to implementation blocks interface shall access all available blocks to provide the API functionality

Chapter 4. Core Driver Usage Model

4.1. Description

In order to support multiple instances and provide the minimum name space pollution, the Core Driver is implemented as a structure that contains a function pointer table.

A public header is provided to define the API object structure and the data structures used. The file is contained in the include directory of the distribution as hpnfc.h.

All the API requires a private data pointer that provides the driver with instance information. It is critical that the upper software layer abides by the programming model.

The client code must obtain the object pointer to the core driver. It can be retrieved by issuing *HPNFC_GetInstance()* function.

The Cadence HPNFC Flash Controller driver will expose the functionality of the hpnfc flash controller using a unified Cadence Driver Interface API. The driver code uses the Cadence Platform Services and therefore will require the presence of CPS for the desired platform. It is expected that a separate layer would use the driver API to implement an interface for the target application. These can be e.g.: Linux driver Application interface (for bare metal hpnfc flash controller applications)

4.2. Initialization

Before calling initialization API, the client of the core driver must first invoke the *probe* function. The *probe* function returns the dynamic memory requirement for the driver.

The client code then must allocate the memory no smaller than what is requested in the return structure provided by the probe function and use that as the private data for the subsequent initialization and other function calls.

```
    HPNFC_SysReq req;
    HPNFC_Config config;
    HPNFC_OBJ *drv = HPNFC_GetInstance();
    void *privateData;
    uint32_t regsBase = 0xfd100000;

    memset(&req, 0, sizeof(HPNFC_SysReq));

    status = drv->probe(regsBase, &req);
    if (status){
        /* initialization failed at this point and should be aborted*/
log("probe failed\n");
return status;
    }

    config.regBase = regsBase;
    /* set handler to callback function using to handle Slave DMA transfer */
    config.slaveDmaHandler = slaveDmaHandler;
    /* number of NAND flash devices connected to the controller */
    config.devNum = 2;
    privateData = malloc(req.pDataSize);

    status = drv->init(privateData, &config);
    if (status){
        printf("Driver initialzation failed\n");
        free(privateData);
```

```
return status;
   }
```

4.3. NAND flash memory command exection

All HPNFC controller command engine operations are defined by *HPNFC_CmdReq* object and executed by *execCmdReq* function. User needs to prepare command request (*HPNFC_CmdReq*) properly and pass it to *execCmdReq* function. There are four types of command requests

- The Command DMA
- The PIO
- The Generic
- Reset Thread

Command request contains few fields common to all requests types. It also contains specific requests for each of above types of request. Depends on request type the proper specific requests should be prepared.

Table 4.1. Specific request depends on request type

request type	specific request path
HPNFC_REQ_TYPE_CDMA	HPNFC_CmdReq.req.cdma
HPNFC_REQ_TYPE_PIO	HPNFC_CmdReq.req.pio
HPNFC_REQ_TYPE_GEN	HPNFC_CmdReq.req.generic
HPNFC_REQ_TYPE_RST	HPNFC_CmdReq.req.reset

After execute *execCmdReq* function, user should monitor *HPNFC_CmdReq.status* until it is not HPNFC_STATUS_PENDING. After that the status field definied in *HPNFC_CmdReq* object contains status of current operation. Status is 0 when operation finished with no error.

4.3.1. Command DMA mode

To execute command in CDMA work mode. User needs to create HPNFC_CdmaCmd object which define CDMA command. Object is necessary to assemble CDMA descriptor. To create one descriptor one *HPNFC_CdmaCmd* command is needed. When descriptor is prepared *HPNFC_CmdReq* object should be filled, Sample preparing CDMA command:

```
HPNFC_CdmaCmd cdmaCmd;
HPNFC_CmdReq cmdReq;
uint8_t thread;
uint32_t cdmaDesc[HPNFC_CDMA_DESC_SIZE / 4];
.
.
.
memset(&cdmaCmd, 0, sizeof(HPNFC_CdmaCmd));

cdmaCmd.bank = 0;
cdmaCmd.rowAddr = 0;
cdmaCmd.cmdCount = 1;
cdmaCmd.cmdType = HPNFC_CDMA_CMD_TYPE_ERASE;

/* assemble CDMA descriptor base on CDMA command */
```

```
drv->assembleCdmaDesc(&cdmaDesc[0], &cdmaCmd);

/* prepare command request */
memset(&cmdReq, 0, sizeof(cmdReq));
cmdReq.req.cdma.descBuf = &cdmaDesc[0];
cmdReq.req.cdma.descDmaAddr = (uintptr_t)&cdmaDesc[0];
cmdReq.hread = thread;
cmdReq.reqType = HPNFC_REQ_TYPE_CDMA;
```

One request can contain a chain of descriptors. To execute such request just put address of first descriptor in the chain to command request object.

4.3.2. PIO mode

Functionality of this work mode is similar to the CDMA work model when descriptors list contains only single descriptor. Following commands are allowed in this work mode:

- Page read - *HPNFC_PIO_CMD_TYPE_READ*
- Page program - *HPNFC_PIO_CMD_TYPE_PROGRAM*
- Erase block - *HPNFC_PIO_CMD_TYPE_ERASE*
- Copy back - *HPNFC_PIO_CMD_TYPE_COPY_BACK*
- Reset - *HPNFC_PIO_CMD_TYPE_RESET*
- Set features - *HPNFC_PIO_CMD_TYPE_SET_FEATURES*

To execute command request common fields and PIO releated field of *HPNFC_CmdReq* object should be filled. Sample preparing PIO command:

```
HPNFC_CmdReq cmdReq;
HPNFC_PioReq *req = &cmdReq.req.pio;
int status;

/* fill common fields */
cmdReq.thread = 0;
cmdReq.reqType = HPNFC_REQ_TYPE_PIO;
cmdReq.status = HPNFC_STATUS_PENDING;

/* fill pio specific fields */
req->rowAddr = 0;
req->bank = 0;
req->volume = 0;
req->cmdType = HPNFC_PIO_CMD_TYPE_ERASE;
```

4.3.3. Generic mode

Generic mode allows to control what exactly is sent to the NAND Flash interface. Therefore in this mode more things need to be made manually. To execute command request, the common fields and Generic releated fields of *HPNFC_CmdGen* object should be filled. The *HPNFC_CmdGen* object contains common fields for all generic commands. It also contains an input which can be different for each of generic command. The input consists of additional values required in command sequence. It can be the command values. address values, number of data bytes to transfer. The input can be divided into five groups.

- Command sequence - used to execute command sequence on the NF interface.
- Address sequence - used to execute address sequence on the NF interface.
- Data sequence - used to transfer the data over the NF interface.
- Read status command - used to Execute 70-sequence +tWHR.
- Address - used for the rest generic commands

Sample preparing Generic command:

```
HPNFC_CmdReq cmdReq;
HPNFC_GenReq *genCmd = &cmdReq.req.generic;

memset(&cmdReq, 0, sizeof(cmdReq));

cmdReq.thread = thread;
cmdReq.reqType = HPNFC_REQ_TYPE_GEN;

genCmd->cmdType = HPNFC_GEN_CMD_TYPE_ERASE;
genCmd->twbActive = 0;
genCmd->input.addr = 0;
genCmd->chipSelect = 0;
```

4.3.4. Operation result

When execution of command request is finished the status field (*HPNFC_CmdReq.status*) of command request should be checked. It is zero when operation succeed. Otherwise it contains error code. In the case when more details about operation is needed the you can get it from HPNFC_OperationResult object. For the PIO and Generic command it is part of request fields. For generic it is located at *HPNFC_CmdReq.req.generic.operationResult.* For PIO request it is located at *HPNFC_CmdReq.req.pio.operationResult* For CDMA request, one command request can contain a chain of descriptors therefore operationResult is get by executing *cdmaDescGetOperationResult*.

4.4. Switching timing work mode

To switch controller and all connected devices to another timing mode the *setWorkMode* function should be executed. Function takes as last parameter PHY configuration *HPNFC_PhyConfig*. This settings should be proper for given timing mode. The PHY configuration is set during changing timing work mode.

4.5. ECC configuration

ECC configuration can be made using two functions there are:

- *setTransferConfig* - used to configure sector size
- *setEccConfig* - used to configure correction strength, enabling/disabling ECC engine etc.

Make sure to use only such ECC strength and sector size configuration which are supported be the controller.

4.6. Device layout configuration

The Controller identifies the device features correctly and sets up device layout. So mainly device layout will be read from controller registers. However, for raw NAND devices where there is no universally accepted protocol of defining the identification sequence, it is possible that the device is not identified correctly. Then it will be necessary to part

of set device layout. Function to read device layout configuration if *getDeviceLayout*. To set part of layout which controller needs to proper work is *setDeviceSubLayout* function.

Chapter 5. Configuration and Hardware Operation Information

5.1. Introduction

The following definitions specify the driver operation environment that is defined by hardware configuration or client code.

These defines are located in the header file of the core driver.

Table 5.1. Configuration and Hardware Operation Information

#define	value	description
HPNFC_CDMA_DESC_SIZE	56U	size of descriptor in bytes
HPNFC_CDMA_DESC_WITH_DUMMY_SIZE	64U	Size is in bytes
HPNFC_STATUS_PENDING	255U	current operation not finished yet

Chapter 6. Dynamic Data Structures

6.1. Introduction

This section defines the data structures used by the driver to provide hardware information, modification and dynamic operation of the driver.

These data structures are defined in the header file of the core driver and utilized by the API.

6.2. HPNFC_ControllerInfo_s

Type: struct

Description

Structure defining all configuration parameters applied on driver start-up.

Fields

`ctrlRev`	Controller revision number.
`ctrlFix`	Fixed number (minor revision number).
`support16b`	Support for 16b NF interface.
`ddr2Support`	Support for NV-DDR2/3 work mode.
`ddrSupport`	Support for NV-DDR work mode.
`asyncSupport`	Support for asynchronous work mode.
`banksNumber`	Maximum number of banks supported by hardware.
	This is an encoded value.
`dmaData64bit`	Slave and Master DMA data width is 64 bit.
`dmaAddr64bit`	Slave and Master DMA address width is 64 bit.
`eccAvailable`	Data ECC protection engine present.
`bootAvailable`	Boot feature present.
`preFetch`	Availability of pre-fetching mechanism.
`dataIntergity`	Availability of Data Integrity mechanism.
`extCmdCnt`	Availability Extended Command Feature.
`remap`	Availability of remap mechanism.
`extStatus`	Availability Extended Status feature.
`dummyDescRead`	Availability of dummy descriptor read feature.

`threadsNumber` Number of threads available in the controller.

6.3. HPNFC_BchConfigInfo_s

Type: struct

Description

Structure defines bch configuration.

Fields

`corrCaps` Supported BCH correction capabilities.

Is sorted from smallest to biggest correction capabilities.

`numberOfCorrCaps` Supported BCH correction capabilities count.

`sectorSizes` Supported BCH sector sizes.

6.4. HPNFC_Config_s

Type: struct

Description

Structure defining all configuration parameters applied on driver start-up.

Fields

`regBase` base address of HPNFC registers

`slaveDmaHandler` Slave DMA handler, passes info about data size and thread number.

`devNum` NAND flash device number connected to controller.

`bchCfgInfo` HW configuration of BCH, it is used only if driver cannot obtain this information from controller registers.

6.5. HPNFC_LunInterleavedConfig_s

Type: struct

Description

Structure defines interleaved operation configuration.

Fields

`lunAddrOff` Bit in ROW address used for selecting of the LUN.

`lunStatSelect`	LUN status select.
`programAferRead`	This bit informs the controller if the device supports a program operation on a LUN while a Read operation is already ongoing in the other LUN.
`multiLunEn`	enable multi LUN interleaved operations

6.6. HPNFC_DeviceSubLayout_s

Type: struct

Description

Structure defines NAND flash memory sublayout.

Only this part of layout which is writable.

Fields

`lunNumber`	The number of LUN presents in the device. Up to 8 LUNs are supported.
`pagesPerBlock`	pages per block - number of pages in a block

6.7. HPNFC_DeviceLayout_s

Type: struct

Description

Structure defines NAND flash memory layout.

Fields

`mainSize`	Main area size in bytes for the NF device page.
`spareSize`	Spare area size in bytes for the NF device page.
`lunNumber`	The number of LUN presents in the device. Up to 8 LUNs are supported.
`pagesPerBlock`	pages per block - number of pages in a block
`blocksPerLun`	number of blocks per LUN
`numberOfPlanes`	number of planes

6.8. HPNFC_StatusPollingConfig_s

Type: struct

Description

Status polling configuration.

Fields

`longPolling`	Number of system clock cycles after issue of erase/write/ read operation before the controller starts to poll for status. This values is valid only in the status polling mode.
`shortPolling`	In the status polling mode, sets the number of cycles the controller must wait before checking the status register.
`readyMask`	the mask used to comparison of response from the NF memory to the device status command and readyValue.
`readyValue`	value which will be compared to response from the NF memory to the device status command
`disableStatusPolling`	Selects R/B pin checks (if 1) or status polling mode (if 0).

6.9. HPNFC_AutoStatusConfig_s

Type: struct

Description

Auto status configurations.

It is used to automatic read and interpret a device status by controller

Fields

`errorMask`	the mask used to comparison of response from the NF memory to the operation status command and status_value
`errorValue`	value which will be compared to response from the NF memory to the operation status command

6.10. HPNFC_TransferConfig_s

Type: struct

Description

Structure defines transfer configuration for program and read pages operations.

Fields

`offset`	Offset value from the beginning of the page, used for data transfer.
`sectorCount`	Numbers of sectors which will be transferred within single NF device page.

`sectorSize` Size of not-last data sector.

`lastSectorSize` Size of last data sector.

6.11. HPNFC_MultiplaneConfig_s

Type: struct

Description

Structure contains multiplane operations configuration.

Fields

`mplWrEn` multiplane write enable

`mplRdEn` multiplane read enable

`mplPrgSeq` multiplane program sequence, it can be 0, 1, 2

`mplRdSeq` multiplane read sequence, it can be 0, 1, 2, 4, 6

`mplEraseSeq` multiplane erase sequence, it can be 0, 1

`lastWrCmd` multiplane last write command sequence, it can be 0, 1

`copyBckCmd` multiplane copy back command sequence, it can be 0, 1

`extStatusEn` select status probing mechanism, 0 - summary status for all planes 1 - separate status per each plane

`planesNumber` number of planes per device it can be 1, 2, 4, 8

6.12. HPNFC_CacheConfig_s

Type: struct

Description

Structure contains cache operation configuration.

Fields

`cacheRdEn` cache read command sequences support

`cacheWrEn` cache write command sequences support

6.13. HPNFC_EccConfiguration_s

Type: struct

Description

Structure defines ECC configuration.

Fields

`eccEnabled`	ECC enabled. Used for enabling and disabling ECC check bits generation and correction.
`scramblerEnabled`	scrambler enabled. Used for enabling/disabling scrambler logic in the controller.
`corrCap`	ECC correction capability.
`skipBytesOffset`	bad block marker - skip bytes offset
`skipBytes`	bad block marker - number of bytes to skip, 0 means no bytes will be skipped
`marker`	bad block marker - 16bit value that will be written in the spare area skip bytes.

6.14. HPNFC_EraseDetectConfig_s

Type: struct

Description

Structure defines erase detection configuration.

Fields

`eraseDetectEnable`	erase detection enable
`eraseDetectLevel`	erase detection level

6.15. HPNFC_PhyConfig_s

Type: struct

Description

Structure defining PHY configuration.

Fields

`phyCtrl`	global control settings for the PHY.
`phyTsel`	global control settings for the termination selects for reads.
`phyDqTiming`	the DQ related timing configuration
`phyDqsTiming`	the DQS related timing configuration
`phyGateLpbkCtrl`	gate and loopback control related timing configuration

`phyDllMasterCtrl`	master DLL logic configuration
`phyDllSlaveCtrl`	slave DLL logic configuration
`toggleTimings0`	NV-DDR/ToggleMode/NV-DDR2/NV-DDR3 timings configuration.
`toggleTimings1`	NV-DDR/ToggleMode/NV-DDR2/NV-DDR3 timings configuration.
`asyncToggleTimings`	NV-DDR/ToggleMode/NV-DDR2/NV-DDR3 and SDR timings configuration.
`timings0`	Global timings configuration - register 0.
`timings1`	Global timings configuration - register 1.
`timings2`	Global timings configuration - register 2.
`dllPhyCtrl`	Configuration of the resynchronization of slave DLL of PHY.

6.16. HPNFC_GenReqInputAddress_s

Type: struct

Description

input for address sequence

Fields

`address`	address bytes to send
`addrSize`	number of addres bytes to send

6.17. HPNFC_GenReqInputData_s

Type: struct

Description

input for data sequence

Fields

`corrCap`	correction capability
`lastSectorSize`	number of data to transfer in the last sector.
`sectorSize`	number of data to transfer of the n-1 sectors except the last one
`sectorCount`	defines the number of sectors to transfer within a single sequence
`erasePageDetectEn`	enable erase page detection
`eccEn`	enable ECC support

`scramblerEnabled`	enable ECC scrambler
`transferDirection`	data tranfser direction
`dataPtr`	Data pointer. Used for slave DMA.
`transferredData`	Number of bytes transferred.

6.18. HPNFC_GenReqInput_s

Type: union

Description

Generic request input field.

Fields

`cmdSeq`	input for command sequence
`addrSeq`	input for address sequence
`dataSeq`	input for data sequence
`addr`	input for commands which need only address parameter.
`readStatusF2En`	input for read status command it F2 enable
`seedValue`	input value for Seed Initial command

6.19. HPNFC_OperationResult_s

Type: struct

Description

Structure contains CDMA operation status.

Fields

`erased`	controller detected an erased page in the read transaction
`maxCorrErr`	maximum amount of correction applied to one ECC sector
`status`	status of operation
`errIdx`	operation number where the first error was detected
`planesDeviceError`	Each bit of this field inform about device error status for the corresponding plane.
`planesEccError`	Each bit of this field inform about ECC error status for the corresponding plane.

6.20. HPNFC_GenReq_s

Type: struct

Description

Structure contains PIO request.

Fields

`input`	Additional values required in command sequence.
`chipSelect`	ChipSelect.
	This field informs the minicotroller on which memory device the sequence need to be executed
`jedecSupp`	is command complaint with Jedec spec
`twbActive`	wait for tWB after sending the last CMD/ADDR/DATA in the sequence.
`cmdType`	Destination bank number.
	It used only for copy back operation.
`operationResult`	Result of execution generic request.

6.21. HPNFC_PioReq_s

Type: struct

Description

Structure contains PIO request.

Fields

`dmaMode`	DMA mode selection master or slave.
	If selected command does not require data transfer, this bit will be ignored
`memPtr`	System/host memory address required for Data DMA commands - (page program and page read).
`bank`	Bank number.
	For copy back operation it is source bank number
`rowAddr`	It is device row address.
	For copy back operation it is row address
`volume`	Volume is an appointed address to a NAND Target.

`dstBank`	Destination bank number. It used only for copy back operation.
`dstRowAddr`	It is destination device row address. It used only for copy back operation
`cmdType`	Command type.
`featureAddr`	It is feature address for set feature command.
`featureValue`	It is feature value for set feature command.
`operationResult`	Result of execution PIO request.
`cmdCount`	Command count (for example for read command how many sequential pages should be read).

6.22. HPNFC_RstReq_s

Type: struct

Description

Structure contains PIO request.

Fields

`resetType`	reset type, can be 0 or 1. Reset type 0 is suitable only to reset CDMA
`abortingRequest`	request wich currently is executed on thread to reset it is internaly used field

6.23. HPNFC_CdmaReq_s

Type: struct

Description

Structure contains CDMA chain request.

Fields

`descBuf`	Pointer to first CDMA descriptor (address accessible by CPU).
`descDmaAddr`	Pointer to first CDMA descriptor (address accessible by DMA engine).

6.24. HPNFC_CdmaSyncConfig_s

Type: struct

Description

configuration of CDMA command sync functionality

Fields

`syncBuffAddr`	Address pointer to sync buffer location.
`startValue`	Start value of the sync buffer flag. The value required to start an operation
`endValue`	Value to be stored to the sync buffer flag upon the successful completion of the operation unless the type is set to Increment.
`startCondition`	condition to meet before start executing operation described by descriptor HPNFC_CDMA_SYNC_CFG_SC_EQUAL - the start condition is the sync buffer flag must be equal to the startValue HPNFC_CDMA_SYNC_CFG_SC_GREATER - the start condition is the sync buffer flag must be equal or greater than the startValue
`endOperation`	operation to be made after operation described by descriptor finished HPNFC_CDMA_SYNC_CFG_EO_INCREMENT - the type is to increment the sync buffer flag and the endValue is ignored HPNFC_CDMA_SYNC_CFG_EO_STORE - the type is to store the endValue to the sync buffer flag
`syncEnable`	enable or disable sync functionality. 1 - enable, 0 - disable

6.25. HPNFC_CdmaCmd_s

Type: struct

Description

Structure contains one CDMA command used to assemble one CDMA descriptor.

Fields

`nextPtr`	Next descriptor address. If there is no next descriptor address set to 0.
`dmaMode`	DMA mode selection master or slave. If selected command does not require data transfer, this bit will be ignored
`memPtr`	System/host memory address required for Data DMA commands - (page program and page read).
`bank`	Bank number. For copy back operation it is source bank number

`rowAddr`	It is device row address. For copy back operation it is row address
`volume`	Volume is an appointed address to a NAND Target.
`dstBank`	Destination bank number. It used only for copy back operation.
`dstRowAddr`	It is destination device row address. It used only for copy back operation
`cmdCount`	Command count (for example for read command how many sequential pages should be read).
`cmdType`	Command type.
`flashPtrCont`	If set continue addressing for the flash pointer instead initialize it using value in the system memory.
`memPtrCont`	If set continue addressing for the memory pointer instead initialize it using value in the system memory.
`syncConfig`	sync configuration for this command

6.26. HPNFC_SysReq_s

Type: struct

Description

Structure used for returning memory allocation size for private data and design configuration required by the driver.

Fields

`pDataSize`	size of memory required for HPNFC_PrivateData struct

6.27. HPNFC_WorkModeSpecificRequest_s

Type: union

Description

Union groups all supported requests to one type.

Fields

`cdma`	CDMA request.
`pio`	Pio request.
`generic`	Generic request.

`reset` Reset thread request.

6.28. HPNFC_CmdReq_s

Type: struct

Description

Structure contains request to execute on HPNFC flash controller.

It can be used to execute command in CDMA, PIO, and Generic work modes.

Fields

`status` status of operation

`thread` thread number for command.

`req` contains all supported requests types(CDMA, PIO and generic).

`reqType` Request type.

6.29. HPNFC_DebugReq_s

Type: struct

Description

Structure contains request to get debug data by HPNFC Debug Interface using register interface.

Fields

`wordSel` Field select with word of the debug vector to read.

`synchMode` Debug synchronize mode in which debug data are read.

`data` read debug data

6.30. HPNFC_DiConfig_s

Type: struct

Description

Configuration of data integrity feature.

Fields

`parityType` Field select parity type.

`errorRespEn` Enable error response generation on SFR interface when parity error detected.

1 - enabled, 0- disabled

`crcEn` Enable CRC checking.

`parityEn` Enable parity checking.

6.31. HPNFC_DiErrInjConfig_s

Type: struct

Description

Configuration of error injection.

Fields

`errInjType` Field select error injection type.

`injCtxValue` Inject error for data of context memory.

`injCtxAddr` Inject error for address of context memory.

`injDescRdAddr` Inject error on address for descriptor read.

`injDescWrAddr` Inject error on address for descriptor status/sync flag write.

`injDescVal` Inject error on address for descriptor status/sync flag write.

`injRegVal` Inject error for register read value.

`injDataRdAddr` Inject error on address for read data.

`injDataWrAddr` Inject error on address for write data.

`injDataWrCrc` Inject CRC error for write data path.

`injDataParityRd` Inject parity error for read data path.

`injRmpAddr` Inject error for Remap SPRAM memory address.

`injRmpVal` Inject error for Remap SPRAM memory (write data).

6.32. HPNFC_DmaConfig_s

Type: struct

Description

Configuration of DMA.

Fields

`dmaPackSizeInWords` Number of 64 bit data words which will be transfered after trig level condition is met.

`trigLvlInWords` Trigger level in terms of 64 bit data words.

6.33. HPNFC_RemapRecord_s

Type: struct

Description

single record of remap table

Fields

`logicAddr` logical address

`physAddr` physical address

6.34. HPNFC_PSlcConfig_s

Type: struct

Description

configuration of pSLC

Fields

`blkAddrIdx` bit index at which block address starts inside the row address

`pSlcPrefixEn` enable 0xA2 prefix command sending for the TLC devices working in the pSLC mode

6.35. HPNFC_ZqCalConfig_s

Type: struct

Description

configuration of ZQ calibration

Fields

`toggImplLcal` this bit informs controller what type of recalibration command needs to be sent after recalibration for selected target was enabled and next was disabled

`targetPerPackage` defines how many NAND flash chips are present in single package conected to the controller

`shortCalEnable` The cyclic re-calibration using the ZQ Calibration Short.

`longCalEnable` The cyclic re-calibration using the ZQ Calibration Long.

`tZqcs` A timing window defined for the ZQ Calibration Short in with the NAND flash device need to perform a full calibration and transfer status values.

`tZqcl`	A timing window defined for the ZQ Calibration Long in with the NAND flash device need to perform a full calibration and transfer status values.
`longCalPeriod`	Time value after with the ZQ Calibration Long operation will be repeated.
`shortCalPeriod`	Time value after with the ZQ Calibration Short operation will be repeated.

6.36. HPNFC_TimeoutConfig_s

Type: struct

Description

timeout configuration feature

Fields

`timeout`	timeout value
`timeoutEn`	enable timeout feature

6.37. HPNFC_DevInfo_s

Type: struct

Description

NAND flash memory basic information.

Fields

`manufacturerId`	NAND flash memory manufacturer ID.
`deviceId`	NAND flash memory device ID.
`devType`	NAND flash memory device ID.
`bitsPerCell`	Number of bits per cell.
`devLayout`	NAND flash memory layout.

6.38. HPNFC_TransferDirection

Type: enum

Description

it defines CDMA command type

Values

`HPNFC_DATA_TRANSFER_READ`	Read data transfer.

Value: 0U

`HPNFC_DATA_TRANSFER_WRITE`

Write data transfer.

Value: 1U

6.39. HPNFC_CdmaCmdType

Type: enum

Description

it defines CDMA command type

Values

`HPNFC_CDMA_CMD_TYPE_ERASE`

block erase command

Value: 1U

`HPNFC_CDMA_CMD_TYPE_PROGRAM`

page program command

Value: 2U

`HPNFC_CDMA_CMD_TYPE_READ` page read command

Value: 3U

`HPNFC_CDMA_CMD_TYPE_COPY_BACK`

copy back command

Value: 4U

`HPNFC_CDMA_CMD_TYPE_RESET`

reset command

Value: 5U

`HPNFC_CDMA_CMD_TYPE_NOP` nop command

Value: 6U

6.40. HPNFC_PioCmdType

Type: enum

Description

it defines PIO command type

Values

`HPNFC_PIO_CMD_TYPE_ERASE` block erase command

Value: 1U

`HPNFC_PIO_CMD_TYPE_PROGRAM`

page program command

Value: 2U

`HPNFC_PIO_CMD_TYPE_READ` page read command

Value: 3U

`HPNFC_PIO_CMD_TYPE_COPY_BACK`

copy back command

Value: 4U

`HPNFC_PIO_CMD_TYPE_RESET` reset command

Value: 5U

`HPNFC_PIO_CMD_TYPE_SET_FEATURES`

nop command

Value: 7U

6.41. HPNFC_GenericCmdType

Type: enum

Description

it defines Generic command type

Values

`HPNFC_GEN_CMD_TYPE_CMD_SEQ`

execute command sequence

Value: 0U

`HPNFC_GEN_CMD_TYPE_ADDR_SEQ`

execute address sequence

Value: 1U

`HPNFC_GEN_CMD_TYPE_DATA_SEQ`

execute data sequence

Value: 2U

`HPNFC_GEN_CMD_TYPE_READ` execute read command

Value: 3U

`HPNFC_GEN_CMD_TYPE_WRITE` execute write command

Value: 4U

`HPNFC_GEN_CMD_TYPE_RESET` execute reset command

Value: 5U

`HPNFC_GEN_CMD_TYPE_ERASE` execute erase command

Value: 6U

`HPNFC_GEN_CMD_TYPE_STATUS`

execute read status command

Value: 7U

`HPNFC_GEN_CMD_TYPE_ENHANCED_STATUS`

execute read enhanced status command

Value: 8U

`HPNFC_GEN_CMD_TYPE_READ_CACHE`

execute read cache random command

Value: 9U

`HPNFC_GEN_CMD_TYPE_COPYBACK_READ`

execute copy back read command

Value: 10U

`HPNFC_GEN_CMD_TYPE_COPYBACK_WRITE`

execute copy back program command

Value: 11U

`HPNFC_GEN_CMD_TYPE_READ_COLUMN`

execute change read column command

Value: 12U

`HPNFC_GEN_CMD_TYPE_READ_COLUMN_ENHANCED`

execute change read column command enhanced

Value: 13U

`HPNFC_GEN_CMD_TYPE_READ_COLUMN_JEDEC`

execute change read column command JEDEC

Value: 14U

`HPNFC_GEN_CMD_TYPE_MULTIPLANE_READ`

execute multiplane read command

Value: 15U

`HPNFC_GEN_CMD_TYPE_MULTIPLANE_ERASE`

execute multiplane erase command

Value: 16U

`HPNFC_GEN_CMD_TYPE_MULTIPLANE_ERASE2`

execute multiplane erase command ONFI JEDEC

Value: 17U

`HPNFC_GEN_CMD_TYPE_WRITE_COLUMN`

execute change write collumn command

Value: 18U

`HPNFC_GEN_CMD_TYPE_SMALL_DATA_MOVE`

execute small data move command

Value: 19U

`HPNFC_GEN_CMD_TYPE_SYNCHRONOUS_RESET`

execute synchronous reset

Value: 20U

`HPNFC_GEN_CMD_TYPE_VOLUME_SELECT`

execute volume select

Value: 21U

`HPNFC_GEN_CMD_TYPE_ODT_CONFIGURE`

execute ODT configure

Value: 22U

`HPNFC_GEN_CMD_TYPE_SET_FEATURES`

execute set features command

Value: 23U

`HPNFC_GEN_CMD_TYPE_GET_FEATURES`

execute get features command

Value: 24U

`HPNFC_GEN_CMD_TYPE_LUN_GET_FEATURES`

execute LUN get features command

Value: 25U

`HPNFC_GEN_CMD_TYPE_LUN_SET_FEATURES`

execute LUN set features command

Value: 26U

`HPNFC_GEN_CMD_TYPE_READ_ID`

execute read ID

Value: 27U

`HPNFC_GEN_CMD_TYPE_READ_PARAM_PAGE`

execute read parameter page

Value: 28U

`HPNFC_GEN_CMD_TYPE_ZQ_CALIB_SHORT`

execute ZQ calibration short

Value: 29U

`HPNFC_GEN_CMD_TYPE_ZQ_CALIB_LONG`

execute ZQ calibration long

Value: 30U

`HPNFC_GEN_CMD_LUN_RESET` execute LUN reset

Value: 31U

`HPNFC_GEN_CMD_SEED_INIT` execute Seed Initial

Value: 32U

6.42. HPNFC_WorkMode

Type: enum

Description

it defines timing work mode

Values

HPNFC_WORK_MODE_SDR	sdr mode (asynchronous)
	Value: 1U
HPNFC_WORK_MODE_NV_DDR	NV DDR mode.
	Value: 2U
HPNFC_WORK_MODE_NV_DDR2	NV DDR2 mode.
	Value: 3U
HPNFC_WORK_MODE_NV_DDR3	NV DDR3 command.
	Value: 4U
HPNFC_WORK_MODE_TOGG	toggle mode
	Value: 5U

6.43. HPNFC_DmaMode

Type: enum

Description

it defines DMA mode

Values

HPNFC_DMA_MODE_SLAVE	Slave DMA mode.
	Value: 0U
HPNFC_DMA_MODE_MASTER	Master DMA mode.
	Value: 1U

6.44. HPNFC_OperationStatus

Type: enum

Description

it defines operation status

Values

`HPNFC_STATUS_NO_ERROR` operation complete without error

Value: 0U

`HPNFC_STATUS_DEV_ERR` device error detected during read status operation

Value: 1U

`HPNFC_STATUS_TIMEOUT_ERR` operation failed to complete successfully because of a time out scenario

Value: 2U

`HPNFC_STATUS_ECC_CORR_ERR`

correctable ECC error was detected.

Value: 3U

`HPNFC_STATUS_ECC_UNCORR_ERR`

uncorrectable ECC error was detected.

Value: 4U

`HPNFC_STATUS_CMD_ERR` invalid command/descriptor was detected

Value: 5U

`HPNFC_STATUS_CMD_UNFINISHED`

command was not executed at all or it was interrupted

Value: 6U

`HPNFC_STATUS_DQS_ERR` Incorrect number of the DQS pulses.

Value: 7U

`HPNFC_STATUS_BUS_ERR` Error response on the system bus.

Value: 8U

`HPNFC_STATUS_PARITY_CTX_ERR`

Parity error during reading from context memory.

Value: 9U

`HPNFC_STATUS_PARITY_DSC_ERR`

Parity error during reading the descriptor.

Value: 10U

`HPNFC_STATUS_CRC_DAT_ERR` Parity or CRC error during data transmission.

Value: 11U

6.45. HPNFC_ReqType

Type: enum

Description

command request type

Values

`HPNFC_REQ_TYPE_CDMA` CDMA request.

Value: 0U

`HPNFC_REQ_TYPE_PIO` PIO request.

Value: 1U

`HPNFC_REQ_TYPE_GEN` Generic request.

Value: 2U

`HPNFC_REQ_TYPE_RST` Thread reset request.

Value: 3U

6.46. HPNFC_LunStatSelect

Type: enum

Description

LUN status select.

Values

`HPNFC_LUN_STAT_SELECT_78` the 78-addr sequence will be use for both first and following LUN

Value: 1U

`HPNFC_LUN_STAT_SELECT_F1_F2`

the F1 command will be used for a first LUN and F2 will be used for second one.

Value: 2U

6.47. HPNFC_CdmaSyncStartCond

Type: enum

Description

sync start condition for CDMA operation

Values

`HPNFC_CDMA_SYNC_CFG_SC_EQUAL`

the start condition is the sync buffer flag must be equal or greater than start value

Value: 1U

`HPNFC_CDMA_SYNC_CFG_SC_GREATER`

the start condition is the sync buffer flag must be equal to the start value

Value: 2U

6.48. HPNFC_CdmaSyncEndOp

Type: enum

Description

operation to be made after operation described by descriptor finished

Values

`HPNFC_CDMA_SYNC_CFG_EO_INCREMENT`

increment the sync buffer flag at the end cdma operation

Value: 1U

`HPNFC_CDMA_SYNC_CFG_EO_STORE`

store the end value to the sync buffer flag at the end cdma operation

Value: 2U

6.49. HPNFC_BootStatus

Type: enum

Description

operation to be made after operation described by descriptor finished

Values

`HPNFC_BOOT_STATUS_OK` no error detected on boot operation

Value: 0U

`HPNFC_BOOT_STATUS_ECC_CORR_ERR`

correctable ECC error was detected

Value: 1U

`HPNFC_BOOT_STATUS_ECC_UNCORR_ERR`

uncorrectable ECC error was detected

Value: 2U

`HPNFC_BOOT_STATUS_AXI_ERR`

DMA master AXI interface receiving an error response from the target.

Value: 3U

`HPNFC_BOOT_STATUS_PHY_ERR`

the boot process fails due to dfi_dqs_underrun error

Value: 4U

`HPNFC_BOOT_STATUS_INIT_ERR`

the boot module finished work without data copy

Value: 5U

6.50. HPNFC_DebugSynchMode

Type: enum

Description

It defines mode in which debug data are read.

Values

`HPNFC_DBG_SYNCH_MODE_NON_SYNCH`

debug data are non synchronized

Value: 1U

`HPNFC_DBG_SYNCH_MODE_SYNCH_BY_NF_CLK`

debug data are synchronized by NF_CLK

Value: 2U

`HPNFC_DBG_SYNCH_MODE_SYNCH_BY_BCH_CLK`

debug data are synchronized by BCH_CLK

Value: 3U

6.51. HPNFC_DebugIfType

Type: enum

Description

It defines Debug access interface type.

Values

`HPNFC_DBG_IF_TYPE_REGISTER`

debug interface is accessible by registers

Value: 1U

`HPNFC_DBG_IF_TYPE_HARDWARE`

debug interface is accessible by hardware

Value: 0U

6.52. HPNFC_DiParityType

Type: enum

Description

It defines parity type for host and SRAM interfaces.

Values

`HPNFC_DI_PARITY_TYPE_EVEN`

even parity type

Value: 1U

`HPNFC_DI_PARITY_TYPE_ODD` odd parity type

Value: 2U

6.53. HPNFC_DiErrInjType

Type: enum

Description

It defines error injection type.

Values

`HPNFC_DI_ERR_INJ_TYPE_ONE_TIME`

one time injection

Value: 1U

`HPNFC_DI_ERR_INJ_TYPE_CONT`

continuous injection

Value: 2U

6.54. HPNFC_PlanesNumber

Type: enum

Description

It defines number of planes.

Values

`HPNFC_PLANES_NUMBER_1` one plane only

Value: 1U

`HPNFC_PLANES_NUMBER_2` two planes

Value: 2U

`HPNFC_PLANES_NUMBER_4` four planes

Value: 4U

`HPNFC_PLANES_NUMBER_8` eight planes

Value: 8U

6.55. HPNFC_MplPrgSeq

Type: enum

Description

It defines multiplane program sequence.

Values

`HPNFC_MPL_PRG_SEQ_0` 80-Addr5-Data-11 repeated for planes 0 to N-2 .

.. 80-Addr5-Data-10 for plane N-1

Value: 0U

`HPNFC_MPL_PRG_SEQ_1` 80-Addr5-Data-11 repeat for planes 0 to N-2.

.. 81-Addr5-Data-10 for plane N-1

Value: 1U

`HPNFC_MPL_PRG_SEQ_2` 80-Addr5-Data-11 or plane 0 .

.. 81-Addr5-Data-11 for planes 1 to N-2... 81-Addr5-Data-10 for plane N-1

Value: 2U

6.56. HPNFC_MplRdSeq

Type: enum

Description

It defines multiplane read sequence.

Values

`HPNFC_MPL_RD_SEQ_0` 00-Addr5_0-32, 00-Addr5_1-32, .

.. ,00-Addr5_n-30, 06-Addr5_0-E0-Data ...

Value: 0U

`HPNFC_MPL_RD_SEQ_1` 00-Addr5/2_0-32, 00-Addr5_1-32, .

.. ,00-Addr5_n-30, 05-Addr5/2_0-E0-Data

Value: 1U

`HPNFC_MPL_RD_SEQ_2` 60-Addr3_0-60-Addr3_1 , .

.., 60-Addr3_n-30, 00-Addr5_0-05-Addr2-E0-DATA ...

Value: 2U

`HPNFC_MPL_RD_SEQ_4` 00-Add5_0-32, 00-Addr5_1-32, .

.. ,00-Addr5_n-30, 00-Addr5_0-05-Addr2-E0 ...

Value: 4U

`HPNFC_MPL_RD_SEQ_6` 60-Addr3_0-60-Addr3_1 , .

.., 60-Addr3_n-C30/C33, 00-Addr5_0-05-Addr2_0-E0-DATA ... 30

Value: 6U

6.57. HPNFC_MplEraseSeq

Type: enum

Description

It defines multiplane erase sequence.

Values

`HPNFC_MPL_ERASE_SEQ_0` ONFI sequence 60-Addr3-d1 60-Addr3-d0.

Value: 0U

`HPNFC_MPL_ERASE_SEQ_1` JEDEC sequence 60-Addr3-60-Addr3-d0.

Value: 1U

6.58. HPNFC_LastWrCmdSeq

Type: enum

Description

It defines multiplane copyback write operation sequence.

Values

`HPNFC_LAST_WR_CMD_SEQ_0` used sequence is 85-Addr5-11 .

.. 85-Addr5-11 ... 85-Addr5-10

Value: 0U

`HPNFC_LAST_WR_CMD_SEQ_1` 85-Addr5-11 .

.. 81-Addr5-11 ... 81-Addr5-10

Value: 1U

6.59. HPNFC_CopyBckCmdSeq

Type: enum

Description

It defines Multiplane copyback read command sequence.

Values

`HPNFC_COPY_BCK_CMD_SEQ_0` used sequence is 00-Addr5-32, .

..., 00-Addr5-35

Value: 0U

`HPNFC_COPY_BCK_CMD_SEQ_1` used sequence is 60-Addr3-60-Addr3-35, 00-Addr5-05-Addr2-e0

Value: 1U

6.60. HPNFC_TargetPerPackage

Type: enum

Description

It defines how many NAND flash chips are present in one package.

Values

`HPNFC_TARGET_PER_PACKAGE_ONE`

single device per package

Value: 1U

`HPNFC_TARGET_PER_PACKAGE_TWO`

two devices per package

Value: 2U

`HPNFC_TARGET_PER_PACKAGE_FOUR`

four devices per package

Value: 4U

`HPNFC_TARGET_PER_PACKAGE_EIGHT`

eight devices per package

Value: 8U

6.61. HPNFC_DevType

Type: enum

Description

It defines NAND flash memory type.

Values

`HPNFC_DEV_TYPE_ONFI`	ONFI compliant device. Value: 1U
`HPNFC_DEV_TYPE_JEDEC`	JEDEC (Toggle) compliant device. Value: 2U

6.62. HPNFC_ControllerInfo

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_ControllerInfo_s` **`HPNFC_ControllerInfo`**

6.63. HPNFC_BchConfigInfo

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_BchConfigInfo_s` **`HPNFC_BchConfigInfo`**

6.64. HPNFC_Config

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_Config_s` **`HPNFC_Config`**

6.65. HPNFC_LunInterleavedConfig

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_LunInterleavedConfig_s` **`HPNFC_LunInterleavedConfig`**

6.66. HPNFC_DeviceSubLayout

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_DeviceSubLayout_s` **`HPNFC_DeviceSubLayout`**

6.67. HPNFC_DeviceLayout

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_DeviceLayout_s` **`HPNFC_DeviceLayout`**

6.68. HPNFC_StatusPollingConfig

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_StatusPollingConfig_s` **`HPNFC_StatusPollingConfig`**

6.69. HPNFC_AutoStatusConfig

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_AutoStatusConfig_s` **`HPNFC_AutoStatusConfig`**

6.70. HPNFC_TransferConfig

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_TransferConfig_s` **`HPNFC_TransferConfig`**

6.71. HPNFC_MultiplaneConfig

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration
`struct HPNFC_MultiplaneConfig_s` **`HPNFC_MultiplaneConfig`**

6.72. HPNFC_CacheConfig

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration
`struct HPNFC_CacheConfig_s` **`HPNFC_CacheConfig`**

6.73. HPNFC_EccConfiguration

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration
`struct HPNFC_EccConfiguration_s` **`HPNFC_EccConfiguration`**

6.74. HPNFC_EraseDetectConfig

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration
`struct HPNFC_EraseDetectConfig_s` **`HPNFC_EraseDetectConfig`**

6.75. HPNFC_PhyConfig

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration
`struct HPNFC_PhyConfig_s` **`HPNFC_PhyConfig`**

6.76. HPNFC_GenReqInputAddress

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration
`struct HPNFC_GenReqInputAddress_s` **`HPNFC_GenReqInputAddress`**

6.77. HPNFC_GenReqInputData

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_GenReqInputData_s` **`HPNFC_GenReqInputData`**

6.78. HPNFC_GenReqInput

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`union HPNFC_GenReqInput_s` **`HPNFC_GenReqInput`**

6.79. HPNFC_OperationResult

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_OperationResult_s` **`HPNFC_OperationResult`**

6.80. HPNFC_GenReq

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_GenReq_s` **`HPNFC_GenReq`**

6.81. HPNFC_PioReq

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_PioReq_s` **`HPNFC_PioReq`**

6.82. HPNFC_RstReq

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_RstReq_s` **`HPNFC_RstReq`**

6.83. HPNFC_CdmaReq

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_CdmaReq_s` **`HPNFC_CdmaReq`**

6.84. HPNFC_CdmaSyncConfig

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_CdmaSyncConfig_s` **`HPNFC_CdmaSyncConfig`**

6.85. HPNFC_CdmaCmd

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_CdmaCmd_s` **`HPNFC_CdmaCmd`**

6.86. HPNFC_SysReq

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_SysReq_s` **`HPNFC_SysReq`**

6.87. HPNFC_WorkModeSpecificRequest

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration
`union HPNFC_WorkModeSpecificRequest_s` **`HPNFC_WorkModeSpecificRequest`**

6.88. HPNFC_CmdReq

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration
`struct HPNFC_CmdReq_s` **`HPNFC_CmdReq`**

6.89. HPNFC_DebugReq

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration
`struct HPNFC_DebugReq_s` **`HPNFC_DebugReq`**

6.90. HPNFC_DiConfig

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration
`struct HPNFC_DiConfig_s` **`HPNFC_DiConfig`**

6.91. HPNFC_DiErrInjConfig

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration
`struct HPNFC_DiErrInjConfig_s` **`HPNFC_DiErrInjConfig`**

6.92. HPNFC_DmaConfig

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration
`struct HPNFC_DmaConfig_s` **`HPNFC_DmaConfig`**

6.93. HPNFC_RemapRecord

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_RemapRecord_s` **`HPNFC_RemapRecord`**

6.94. HPNFC_PSlcConfig

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_PSlcConfig_s` **`HPNFC_PSlcConfig`**

6.95. HPNFC_ZqCalConfig

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_ZqCalConfig_s` **`HPNFC_ZqCalConfig`**

6.96. HPNFC_TimeoutConfig

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_TimeoutConfig_s` **`HPNFC_TimeoutConfig`**

6.97. HPNFC_DevInfo

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

`struct HPNFC_DevInfo_s` **`HPNFC_DevInfo`**

6.98. HPNFC_SlaveDmaHandler

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* HPNFC_SlaveDmaHandler )(void *privateData, void *dataPtr,
HPNFC_TransferDirection transferDirection, uint32_t sdmaSize, uint32_t
*transferredData)
```

Description

Callback for slave dma interrupt handling.

Chapter 7. Driver API Object

7.1. Introduction

API listing for the driver.

The API is contained in the object as function pointers in the object structure. As the actual functions resides in the Driver Object, the client software must first use the global GetInstance function to obtain the Driver Object Pointer. The actual APIs then can be invoked using obj->(api_name)() syntax. These functions are defined in the header file of the core driver and utilized by the API.

7.2. HPNFC_Probe

Function Declaration

`uint32_t HPNFC_Probe(`*`registerBase`*`,` *`req`*`);`

`uintptr_t registerBase`

`HPNFC_SysReq *req`

Parameter List

`registerBase` [in]

base address of controller registers

`req` [out]

on success, filled with the number of bytes the driver requires for private data.

Description

Get the driver's memory requirements to support the configuration of the given controller.

Return Value

0 on success (dataSize filled)

ENOTSUP if configuration cannot be supported due to driver/hardware constraints

7.3. HPNFC_Init

Function Declaration

`int32_t HPNFC_Init(`*`privateData`*`,` *`config`*`);`

`void *privateData`

`HPNFC_Config *config`

Parameter List

`privateData` [in]

pointer to memory allocated for the driver use of sufficient size to hold the number of bytes returned by probe in HPNFC_SysReq

`config` [in]

contains driver and hardware configuration as determined by probe call(s)

Description

The init function makes the following jobs: - initializes the driver instance and state - configures the hardware as specified by the client - initializes hardware.

Return Value

0 on success

EINVAL if config is NULL or if illegal/inconsistent values in config

7.4. HPNFC_Isr

Function Declaration

`void HPNFC_Isr(`*`privateData`*`);`

`void *privateData`

Parameter List

`privateData` [in]

driver instance data filled by init

Description

Driver ISR.

Platform-specific code is responsible for ensuring this gets called when the controller's interrupt is asserted. Registering the ISR should be done after calling init, and before calling start.

7.5. HPNFC_Start

Function Declaration

`void HPNFC_Start(`*`privateData`*`);`

`void *privateData`

Parameter List

`privateData` [in]

driver instance data filled by init

Description

Start the driver.

This enables any interrupts handled by the driver. This is called after the client has successfully initialized the driver and hooked the driver's ISR (the isr member of this struct) to the IRQ.

7.6. HPNFC_Stop

Function Declaration

`void HPNFC_Stop(`*`privateData`*`);`

`void *privateData`

Parameter List

`privateData` [in]

driver instance data filled by init

Description

Stop the driver.

This disables any interrupts being handled by the driver.

7.7. HPNFC_Destroy

Function Declaration

`void HPNFC_Destroy(`*`privateData`*`);`

`void *privateData`

Parameter List

`privateData` [in]

driver instance data filled by init

Description

Destroy the driver (automatically performs a stop).

7.8. HPNFC_AssembleCdmaDesc

Function Declaration

`int32_t HPNFC_AssembleCdmaDesc(`*`privateData`*`, `*`descPtr`*`, `*`cdmaCmd`*`);`

`void *privateData`

`void *descPtr`

`HPNFC_CdmaCmd *cdmaCmd`

Parameter List

`privateData` [in]

driver instance data filled by init

`descPtr` [in]

descriptor pointer

`cdmaCmd` [in]

CDMA command used to assemble one CDMA descriptor

Description

Function assembles CDMA descriptor base on cdmaCmd.

Return Value

0 on success

EINVAL if descPtr is NULL

7.9. HPNFC_ExecCmdReq

Function Declaration

`int32_t HPNFC_ExecCmdReq(`*`privateData`*`, `*`cmdReq`*`);`

`void *privateData`

`HPNFC_CmdReq *cmdReq`

Parameter List

`privateData` [in]

driver instance data filled by init

`cmdReq` [in]

Request to execute

Description

Function executes request.

Return Value

0 on success

EINVAL if cmdReq is NULL or if illegal/inconsistent values in cmdReq

EBUSY if given thread defined in cmdReq is busy

7.10. HPNFC_SetWorkMode

Function Declaration

`int32_t HPNFC_SetWorkMode(`*`privateData`*`, `*`workMode`*`, `*`timingMode`*`, `*`phyConfig`*`);`

```
void *privateData

HPNFC_WorkMode workMode

uint8_t timingMode

HPNFC_PhyConfig *phyConfig
```

Parameter List

`privateData` [in]

driver instance data filled by init

`workMode` [in]

work mode to set up

`timingMode` [in]

timing mode of work mode

`phyConfig` [in]

phy configuration for given work mode

Description

Function change controller and device working mode.

Return Value

0 on success

EINVAL if phyConfig is NULL

EBUSY if command send to memory device to change its mode cannot be finished

7.11. HPNFC_SetEccConfig

Function Declaration

`int32_t HPNFC_SetEccConfig(`*`privateData`*`, `*`eccConfig`*`);`

```
void *privateData
```

```
HPNFC_EccConfiguration *eccConfig
```

Parameter List

`privateData` [in]

driver instance data filled by init

`eccConfig` [in]

new ECC configuration

Description

Function configures ECC module of controller.

Return Value

0 on success

EINVAL if eccConfig is NULL or if illegal/inconsistent values in eccConfig

ENOTSUP if ECC feature is not supported by controller

EBUSY if controller is busy

7.12. HPNFC_GetEccConfig

Function Declaration

`int32_t HPNFC_GetEccConfig(`*`privateData`*`,` *`eccConfig`*`);`

```
void *privateData
```

```
HPNFC_EccConfiguration *eccConfig
```

Parameter List

`privateData` [in]

driver instance data filled by init

`eccConfig` [out]

current ECC configuration

Description

Function get current ECC configuration.

Return Value

0 on success

EINVAL if eccConfig is NULL

ENOTSUP if ECC feature is not supported by controller

7.13. HPNFC_SetTransferConfig

Function Declaration

`int32_t HPNFC_SetTransferConfig(`*`privateData`*`, `*`transferCfg`*`);`

`void *privateData`

`HPNFC_TransferConfig *transferCfg`

Parameter List

`privateData` [in]

driver instance data filled by init

`transferCfg` [in]

new transfer configuration configuration

Description

Function sets transfer configuration for program and read page operations.

Return Value

0 on success

EINVAL if transferCfg is NULL or if illegal/inconsistent values in transferCfg

EBUSY if controller is busy

7.14. HPNFC_GetTransferConfig

Function Declaration

`int32_t HPNFC_GetTransferConfig(`*`privateData`*`, `*`transferCfg`*`);`

`void *privateData`

`HPNFC_TransferConfig *transferCfg`

Parameter List

`privateData` [in]

driver instance data filled by init

`transferCfg` [out]

current transfer configuration configuration

Description

Function gets transfer configuration.

Return Value

0 on success

EINVAL if transferCfg is NULL

EBUSY if controller is busy

7.15. HPNFC_CdmaGetFailedDesc

Function Declaration

`int32_t HPNFC_CdmaGetFailedDesc(`*`cdmaDesc`*`, `*`*`fail`*`);`

`void *cdmaDesc`

`void **fail`

Parameter List

`cdmaDesc` [in]

first descriptor in the chain

`fail` [out]

a pointer to the descriptor whose execution failed

Description

Function can be used to get first failed descritor from chain of descriptors.

Return Value

0 on success

EINVAL if *fail is NULL

7.16. HPNFC_CdmaGetNextFailedDesc

Function Declaration

`int32_t HPNFC_CdmaGetNextFailedDesc(`*`cdmaDesc`*`, `*`*`failed`*`);`

`void *cdmaDesc`

```
void **failed
```

Parameter List

`cdmaDesc` [in]

a descriptor in the chain

`failed` [out]

a pointer to the descriptor whose execution failed

Description

Function can be used to get next failed descriptor.

Current descriptor is not checked. The search begins from the next descriptor.

Return Value

0 on success

EINVAL if *failed is NULL

7.17. HPNFC_CdmaDescGetOperationResult

Function Declaration

`int32_t HPNFC_CdmaDescGetOperationResult(`*`privateData`*`, `*`cdmaDesc`*`, `*`operationResult`*`);`

```
void *privateData
```

```
void *cdmaDesc
```

```
HPNFC_OperationResult *operationResult
```

Parameter List

`privateData` [in]

driver instance data filled by init

`cdmaDesc` [in]

a descriptor to check

`operationResult` [out]

status of operation defined in descriptor

Description

The function returns the detailed status of the CDMA operation.

Return Value

0 on success

EINVAL if operationResult is NULL

7.18. HPNFC_ReqGetOperationResult

Function Declaration

`int32_t HPNFC_ReqGetOperationResult(`*`privateData`*`, `*`cmdReq`*`, `*`operationResult`*`);`

```
void *privateData

HPNFC_CmdReq *cmdReq

HPNFC_OperationResult *operationResult
```

Parameter List

`privateData`	[in]	driver instance data filled by init
`cmdReq`	[in]	a request to check
`operationResult`	[out]	status of operation defined in descriptor

Description

The function returns the detailed status of a operation from HPNFC registers It can be used when the HPNFC controller is not able to update status inside a descriptor because of data integrity error.

Then status can be read from registers instead of reading it from a descriptor.

Return Value

0 on success

EINVAL if operationResult is NULL

7.19. HPNFC_CdmaParseDesc

Function Declaration

`int32_t HPNFC_CdmaParseDesc(`*`privateData`*`, `*`descPtr`*`, `*`cdmaCmd`*`);`

```
void *privateData

void *descPtr

HPNFC_CdmaCmd *cdmaCmd
```

Parameter List

`privateData` [in]

driver instance data filled by init

`descPtr` [in]

descriptor pointer to parse

`cdmaCmd` [out]

CDMA command created base on descriptor pointer

Description

Function create cdmaCmd command base on descriptor.

Return Value

0 on success

EINVAL if cdmaCmd is NULL

7.20. HPNFC_SetStatusPollingConfig

Function Declaration

`int32_t HPNFC_SetStatusPollingConfig(`*`privateData`*`, `*`statusPollingConfig`*`);`

`void *privateData`

`HPNFC_StatusPollingConfig *statusPollingConfig`

Parameter List

`privateData` [in]

driver instance data filled by init

`statusPollingConfig` [in]

new status polling configuration

Description

Function sets status polling configuration.

Return Value

0 on success

EINVAL if statusPollingConfig is NULL

EBUSY if controller is busy

7.21. HPNFC_GetStatusPollingConfig

Function Declaration

`int32_t HPNFC_GetStatusPollingConfig(`*`privateData`*`, `*`statusPollingConfig`*`);`

```
void *privateData

HPNFC_StatusPollingConfig *statusPollingConfig
```

Parameter List

`privateData`	[in] driver instance data filled by init
`statusPollingConfig`	[out] current status polling configuration

Description

Function gets status polling configuration.

Return Value

0 on success

EINVAL if statusPollingConfig is NULL

7.22. HPNFC_SetAutoStatusConfig

Function Declaration

`int32_t HPNFC_SetAutoStatusConfig(`*`privateData`*`, `*`autoStatusConfig`*`);`

```
void *privateData

HPNFC_AutoStatusConfig *autoStatusConfig
```

Parameter List

`privateData`	[in] driver instance data filled by init
`autoStatusConfig`	[in] new status configuration

Description

Function sets auto status configuration.

Return Value

0 on success

EINVAL if autoStatusConfig is NULL

EBUSY if controller is busy

7.23. HPNFC_GetAutoStatusConfig

Function Declaration

`int32_t HPNFC_GetAutoStatusConfig(`*`privateData`*`, `*`autoStatusConfig`*`);`

```
void *privateData

HPNFC_AutoStatusConfig *autoStatusConfig
```

Parameter List

`privateData` [in]

driver instance data filled by init

`autoStatusConfig` [out]

current auto status configuration

Description

Function gets auto status configuration.

Return Value

0 on success

EINVAL if autoStatusConfig is NULL

7.24. HPNFC_GetDeviceLayout

Function Declaration

`int32_t HPNFC_GetDeviceLayout(`*`privateData`*`, `*`deviceLayout`*`);`

```
void *privateData

HPNFC_DeviceLayout *deviceLayout
```

Parameter List

`privateData` [in]

driver instance data filled by init

`deviceLayout` [out]

current device layout

Description

Function gets device layout.

Return Value

0 on success

EINVAL if deviceLayout is NULL

7.25. HPNFC_SetDeviceSubLayout

Function Declaration

`int32_t HPNFC_SetDeviceSubLayout(`*`privateData`*`, `*`deviceSubLayout`*`);`

`void *privateData`

`HPNFC_DeviceSubLayout *deviceSubLayout`

Parameter List

`privateData` [in]

driver instance data filled by init

`deviceSubLayout` [in]

new device sublayout to set

Description

Function sets device sublayout.

Return Value

0 on success

EINVAL if deviceSubLayout is NULL

EBUSY if controller is busy

7.26. HPNFC_SetLunInterleavedConfig

Function Declaration

`int32_t HPNFC_SetLunInterleavedConfig(`*`privateData`*`, `*`lunInterleavedConfig`*`);`

`void *privateData`

```
HPNFC_LunInterleavedConfig *lunInterleavedConfig
```

Parameter List

`privateData`	[in]	driver instance data filled by init
`lunInterleavedConfig`	[in]	new interleaved configuration to set

Description

Function sets LUN interleaved configuration.

Return Value

0 on success

EINVAL if lunInterleavedConfig is NULL if illegal/inconsistent values in lunInterleavedConfig

EBUSY if controller is busy

7.27. HPNFC_GetLunInterleavedConfig

Function Declaration

`int32_t HPNFC_GetLunInterleavedConfig(`*`privateData`*`, `*`lunInterleavedConfig`*`);`

```
void *privateData
```

```
HPNFC_LunInterleavedConfig *lunInterleavedConfig
```

Parameter List

`privateData`	[in]	driver instance data filled by init
`lunInterleavedConfig`	[out]	current interleaved configuration

Description

Function gets LUN interleaved configuration.

Return Value

0 on success

EINVAL if lunInterleavedConfig is NULL

7.28. HPNFC_GetRnb

Function Declaration

`int32_t HPNFC_GetRnb(`*`privateData`*`, `*`chip`*`, `*`rnb`*`);`

```
void *privateData

uint8_t chip

uint8_t *rnb
```

Parameter List

`privateData`	[in]	driver instance data filled by init
`chip`	[in]	chip number to check rnb signal
`rnb`	[out]	current chip Ready/Busy# signal state

Description

Function gets current Ready/Busy# line status of one chip.

Return Value

0 on success

EINVAL if rnb is NULL

7.29. HPNFC_GetBootStatus

Function Declaration

`int32_t HPNFC_GetBootStatus(`*`privateData`*`, `*`bootStatus`*`, `*`cpyId`*`);`

```
void *privateData

HPNFC_BootStatus *bootStatus

uint8_t *cpyId
```

Parameter List

`privateData`	[in]	driver instance data filled by init
`bootStatus`	[out]	

boot status

`cpyId` [out]

boot block used in the latest boot sequence run

Description

Function gets status of boot operation.

Return Value

0 on success

EINVAL if cpyId is NULL

ENOTSUP if boot feature is not supported by controller

7.30. HPNFC_GetMultiplaneConfig

Function Declaration

`int32_t HPNFC_GetMultiplaneConfig(`*`privateData`*`,`*`multiplaneConfig`*`);`

`void *privateData`

`HPNFC_MultiplaneConfig *multiplaneConfig`

Parameter List

`privateData` [in]

driver instance data filled by init

`multiplaneConfig` [out]

current multiplane configuration

Description

Function gets multiplane configuration.

Return Value

0 on success

EINVAL if multiplaneConfig is NULL

7.31. HPNFC_SetMultiplaneConfig

Function Declaration

`int32_t HPNFC_SetMultiplaneConfig(`*`privateData`*`,`*`multiplaneConfig`*`);`

```
void *privateData

HPNFC_MultiplaneConfig *multiplaneConfig
```

Parameter List

`privateData`	[in]	driver instance data filled by init
`multiplaneConfig`	[in]	new multiplane configuration to set

Description

Function sets multiplane operations configuration.

Return Value

0 on success

EINVAL if multiplaneConfig is NULL if illegal/inconsistent values in multiplaneConfig

EBUSY if controller is busy

7.32. HPNFC_GetCacheConfig

Function Declaration

`int32_t HPNFC_GetCacheConfig(`*`privateData`*`, `*`cacheConfig`*`);`

```
void *privateData

HPNFC_CacheConfig *cacheConfig
```

Parameter List

`privateData`	[in]	driver instance data filled by init
`cacheConfig`	[out]	current cache configuration

Description

Function gets cache operations configuration.

Return Value

0 on success

EINVAL if cacheConfig is NULL

7.33. HPNFC_SetCacheConfig

Function Declaration

`int32_t HPNFC_SetCacheConfig(`*`privateData`*`, `*`cacheConfig`*`);`

```
void *privateData

HPNFC_CacheConfig *cacheConfig
```

Parameter List

`privateData` [in]

driver instance data filled by init

`cacheConfig` [in]

new cache configuration

Description

Function sets cache operations configuration.

Return Value

0 on success

EINVAL if cacheConfig is NULL

EBUSY if controller is busy

7.34. HPNFC_GetBchConfigInfo

Function Declaration

`int32_t HPNFC_GetBchConfigInfo(`*`privateData`*`, `*`bchConfigInfo`*`);`

```
void *privateData

HPNFC_BchConfigInfo *bchConfigInfo
```

Parameter List

`privateData` [in]

driver instance data filled by init

`bchConfigInfo` [out]

BCH supported configurations

Description

Function gets ECC BCH configuration info (supported features).

Return Value

0 on success

EINVAL if bchConfigInfo is NULL

7.35. HPNFC_GetControllerInfo

Function Declaration

`int32_t HPNFC_GetControllerInfo(`*`privateData`*`, `*`controllerInfo`*`);`

```
void *privateData
```

```
HPNFC_ControllerInfo *controllerInfo
```

Parameter List

`privateData` [in]

driver instance data filled by init

`controllerInfo` [out]

Controller supported features

Description

Function gets controller info (supported features).

Return Value

0 on success

EINVAL if controllerInfo is NULL

7.36. HPNFC_GetCdmaDescSize

Function Declaration

`int32_t HPNFC_GetCdmaDescSize(`*`privateData`*`, `*`descSize`*`);`

```
void *privateData
```

```
uint8_t *descSize
```

Parameter List

`privateData` [in]

driver instance data filled by init

`descSize` [out]

descriptor size in bytes

Description

Function gets CDMA descriptor size in bytes which is expected by hardware.

Return Value

0 on success

EINVAL if descSize is NULL

7.37. HPNFC_GetDebugData

Function Declaration

`int32_t HPNFC_GetDebugData(`*`privateData`*`, `*`dbgReq`*`);`

`void *privateData`

`HPNFC_DebugReq *dbgReq`

Parameter List

`privateData` [in]

driver instance data filled by init

`inout]` [ ]

dbgReq debug request to be executed on Debug Interface

Description

Function gets debug data using HPNFC Debug Interface.

Return Value

0 on success

EINVAL if dbgReq is NULL or if illegal/inconsistent values in dbgReq

EIO if register debug interface is not selected

ENOTSUP if HPNFC controller does not support debug feature

7.38. HPNFC_SetDebugIfType

Function Declaration

`int32_t HPNFC_SetDebugIfType(`*`privateData`*`, `*`ifType`*`);`

`void *privateData`

```
HPNFC_DebugIfType ifType
```

Parameter List

`privateData` [in]

driver instance data filled by init

`ifType` [in]

debug nterface type

Description

Function sets Debug Interface type.

Return Value

0 on success

EINVAL if ifType has wrong value

ENOTSUP if HPNFC controller does not support debug feature

7.39. HPNFC_GetDebugIfType

Function Declaration

`int32_t HPNFC_GetDebugIfType(`*`privateData`*`,` *`ifType`*`);`

```
void *privateData
```

```
HPNFC_DebugIfType *ifType
```

Parameter List

`privateData` [in]

driver instance data filled by init

`ifType` [out]

debug interface type

Description

Function gets Debug Interface type.

Return Value

0 on success

EINVAL if ifType has wrong value

ENOTSUP if HPNFC controller does not support debug feature

7.40. HPNFC_SetDiConfig

Function Declaration

`int32_t HPNFC_SetDiConfig(`*`privateData`*`,` *`diConfig`*`);`

`void *privateData`

`HPNFC_DiConfig *diConfig`

Parameter List

`privateData` [in]

driver instance data filled by init

`diConfig` [in]

data integrity configuration

Description

Function sets data integrity configuration.

Return Value

0 on success

EINVAL if diConfig is NULL or if illegal/inconsistent values in diConfig

ENOTSUP if HPNFC controller does not support data integrity feature

EBUSY if controller is busy

7.41. HPNFC_GetDiConfig

Function Declaration

`int32_t HPNFC_GetDiConfig(`*`privateData`*`,` *`diConfig`*`);`

`void *privateData`

`HPNFC_DiConfig *diConfig`

Parameter List

`privateData` [in]

driver instance data filled by init

`diConfig` [out]

data integrity configuration

Description

Function current data integrity configuration.

Return Value

0 on success

EINVAL if privateData or diConfig is NULL

ENOTSUP if HPNFC controller does not support data integrity feature

7.42. HPNFC_SetDiErrInjConfig

Function Declaration

`int32_t HPNFC_SetDiErrInjConfig(`*`privateData`*`, `*`diErrInjConfig`*`);`

`void *privateData`

`HPNFC_DiErrInjConfig *diErrInjConfig`

Parameter List

`privateData` [in]

driver instance data filled by init

`diErrInjConfig` [in]

error injection configuration to set

Description

Function sets error injection configuration.

Return Value

0 on success

EINVAL if diErrInjConfig is NULL or if illegal/inconsistent values in diErrInjConfig

ENOTSUP if HPNFC controller does not support data integrity feature

EBUSY if controller is busy

7.43. HPNFC_GetDiErrInjConfig

Function Declaration

`int32_t HPNFC_GetDiErrInjConfig(`*`privateData`*`, `*`diErrInjConfig`*`);`

`void *privateData`

`HPNFC_DiErrInjConfig *diErrInjConfig`

Parameter List

`privateData` [in]

driver instance data filled by init

`diErrInjConfig` [out]

read error injection configuration

Description

Function gets error injection configuration.

Return Value

0 on success

EINVAL if diErrInjConfig is NULL or if illegal/inconsistent values in diErrInjConfig

ENOTSUP if HPNFC controller does not support data integrity feature

7.44. HPNFC_SetDmaConfig

Function Declaration

`int32_t HPNFC_SetDmaConfig(`*`privateData`*`,` *`dmaConfig`*`);`

```
void *privateData
```

```
HPNFC_DmaConfig *dmaConfig
```

Parameter List

`privateData` [in]

driver instance data filled by init

`dmaConfig` [in]

DMA configuration

Description

Function sets DMA configuration.

Return Value

0 on success

EINVAL if dmaConfig is NULL

ENOTSUP if HPNFC controller does not support configuration of DMA

7.45. HPNFC_GetDmaConfig

Function Declaration

`int32_t HPNFC_GetDmaConfig(`*`privateData`*`, `*`dmaConfig`*`);`

```
void *privateData
```

```
HPNFC_DmaConfig *dmaConfig
```

Parameter List

`privateData` [in]

driver instance data filled by init

`dmaConfig` [out]

DMA configuration

Description

Function gets DMA configuration.

Return Value

0 on success

EINVAL if dmaConfig is NULL

ENOTSUP if HPNFC controller does not support configuration of DMA

7.46. HPNFC_WriteRemapRecord

Function Declaration

`int32_t HPNFC_WriteRemapRecord(`*`privateData`*`, `*`remapRecord`*`, `*`memTrg`*`);`

```
void *privateData
```

```
HPNFC_RemapRecord *remapRecord
```

```
uint8_t memTrg
```

Parameter List

`privateData` [in]

driver instance data filled by init

`remapRecord` [in]

remap record to write

`in` [ ]

determines which memory target is bind to current write] memTrg

Description

Function writes single record of remap table.

Return Value

0 on success

EINVAL if remapRecord is NULL

ENOTSUP if HPNFC controller does not support remap mechanism

7.47. HPNFC_ReadRemapRecord

Function Declaration

`int32_t HPNFC_ReadRemapRecord(`*`privateData`*`, `*`recordIdx`*`, `*`remapRecord`*`, `*`memTrg`*`);`

```
void *privateData

uint16_t recordIdx

HPNFC_RemapRecord *remapRecord

uint8_t memTrg
```

Parameter List

`privateData` [in]

driver instance data filled by init

`recordIdx` [in]

index of remap record

`remapRecord` [out]

read remap record

`in` [ ]

determines which memory target is bind to current read] memTrg

Description

Function reads single record of remap table.

Return Value

0 on success

EINVAL if remapRecord is NULL

ENOTSUP if HPNFC controller does not support remap mechanism

7.48. HPNFC_ClearRemapTable

Function Declaration

`int32_t HPNFC_ClearRemapTable(`*`privateData`*`);`

`void *privateData`

Parameter List

`privateData` [in]

driver instance data filled by init

Description

Function clears remap table.

Return Value

0 on success

EINVAL if privateData is NULL

ENOTSUP if HPNFC controller does not support remap mechanism

7.49. HPNFC_GetRemapRecordCount

Function Declaration

`int32_t HPNFC_GetRemapRecordCount(`*`privateData`*`, `*`recordCount`*`);`

`void *privateData`

`uint16_t *recordCount`

Parameter List

`privateData` [in]

driver instance data filled by init

`recordCount` [out]

number of records stored in remap table

Description

Function gets number of records stored in remap table.

Return Value

0 on success

EINVAL if recordCount is NULL

ENOTSUP if HPNFC controller does not support remap mechanism

7.50. HPNFC_SetRemapMask

Function Declaration

`int32_t HPNFC_SetRemapMask(`*`privateData`*`, `*`remapMask`*`);`

```
void *privateData
```

```
uint32_t remapMask
```

Parameter List

`privateData` [in]

driver instance data filled by init

`remapMask` [in]

remap mask to be set

Description

Function sets remap mask.

Return Value

0 on success

EINVAL if privateData is NULL

ENOTSUP if HPNFC controller does not support remap mechanism

7.51. HPNFC_GetRemapMask

Function Declaration

`int32_t HPNFC_GetRemapMask(`*`privateData`*`, `*`remapMask`*`);`

```
void *privateData
```

```
uint32_t *remapMask
```

Parameter List

`privateData` [in]

driver instance data filled by init

`remapMask` [out]

remap mask to be set

Description

Function gets remap mask.

Return Value

0 on success

EINVAL if remapMask is NULL

ENOTSUP if HPNFC controller does not support remap mechanism

7.52. HPNFC_SetRemapEnable

Function Declaration

`int32_t HPNFC_SetRemapEnable(`*`privateData`*`, `*`remapEnable`*`);`

`void *privateData`

`uint8_t remapEnable`

Parameter List

`privateData` [in]

driver instance data filled by init

`remapEnable` [in]

configuration of remap, 0 - disabled, 1 - enabled

Description

Function sets remap enable.

Return Value

0 on success

EINVAL if privateData is NULL

ENOTSUP if HPNFC controller does not support remap mechanism

7.53. HPNFC_GetRemapEnable

Function Declaration

`int32_t HPNFC_GetRemapEnable(`*`privateData`*`, `*`remapEnable`*`);`

`void *privateData`

`uint8_t *remapEnable`

Parameter List

`privateData` [in]

driver instance data filled by init

`remapEnable` [out]

configuration of remap, 0 - disabled, 1 - enabled

Description

Function gets remap enable.

Return Value

0 on success

EINVAL if remapEnable is NULL

ENOTSUP if HPNFC controller does not support remap mechanism

7.54. HPNFC_SetPSlcConfig

Function Declaration

`int32_t HPNFC_SetPSlcConfig(`*`privateData`*`,` *`pSlcConfig`*`);`

`void *privateData`

`HPNFC_PSlcConfig *pSlcConfig`

Parameter List

`privateData` [in]

driver instance data filled by init

`pSlcConfig` [in]

configuration pSLC to set

Description

Function sets pSLC configuration.

Return Value

0 on success

EINVAL if pSlcConfig.pSlcPrefixEn > 1

ENOTSUP if HPNFC controller does not support pSLC feautre

7.55. HPNFC_GetPSlcConfig

Function Declaration

`int32_t HPNFC_GetPSlcConfig(`*`privateData`*`, `*`pSlcConfig`*`);`

`void *privateData`

`HPNFC_PSlcConfig *pSlcConfig`

Parameter List

`privateData` [in]

driver instance data filled by init

`pSlcConfig` [out]

current pSlc configuration

Description

Function gets pSLC configuration.

Return Value

0 on success

EINVAL if privateData is NULL

ENOTSUP if HPNFC controller does not support pSLC feautre

7.56. HPNFC_SetZqCalConfig

Function Declaration

`int32_t HPNFC_SetZqCalConfig(`*`privateData`*`, `*`zqCalConfig`*`);`

`void *privateData`

`HPNFC_ZqCalConfig *zqCalConfig`

Parameter List

`privateData` [in]

driver instance data filled by init

`zqCalConfig` [in]

configuration of ZQ calibration to set

Description

Function sets ZQ calbration configuration.

Return Value

0 on success

EINVAL if zqCalConfig is NULL or if illegal/inconsistent values in zqCalConfig

7.57. HPNFC_GetZqCalConfig

Function Declaration

`int32_t HPNFC_GetZqCalConfig(`*`privateData`*`, `*`zqCalConfig`*`);`

`void *privateData`

`HPNFC_ZqCalConfig *zqCalConfig`

Parameter List

`privateData` [in]

driver instance data filled by init

`zqCalConfig` [out]

current configuration of ZQ calibration

Description

Function gets ZQ calbration configuration.

Return Value

0 on success

EINVAL if zqCalConfig is NULL

7.58. HPNFC_SetZqCalEnable

Function Declaration

`int32_t HPNFC_SetZqCalEnable(`*`privateData`*`, `*`target`*`, `*`enable`*`);`

`void *privateData`

`uint8_t target`

`uint8_t enable`

Parameter List

`privateData` [in]

driver instance data filled by init

`target` [in]

target number - bank or volume, depending on HPNFC configuration

`enable` [in]

parameter is used to enable or disable ZQ calibration functionality 1 - enable, 0 - disable

Description

Function sets ZQ calbration enable.

Return Value

0 on success

EINVAL if enable is bigger than 1

7.59. HPNFC_GetZqCalEnable

Function Declaration

`int32_t HPNFC_GetZqCalEnable(`*`privateData`*`, `*`target`*`, `*`enable`*`);`

```
void *privateData
```

```
uint8_t target
```

```
uint8_t *enable
```

Parameter List

`privateData` [in]

driver instance data filled by init

`target` [in]

target number - bank or volume, depending on HPNFC configuration

`enable` [out]

iforms whether ZQ calibration is enabled or disabled 1 - enabled, 0 - disabled

Description

Function gets ZQ calbration enable.

Return Value

0 on success

EINVAL if target is bigger than 15

7.60. HPNFC_GetZqStat

Function Declaration

`int32_t HPNFC_GetZqStat(`*`privateData`*`, `*`calStat`*`);`

`void *privateData`

`uint16_t *calStat`

Parameter List

`privateData` [in]

driver instance data filled by init

`calStat` [out]

Each bit of calStat informs about ZQ recalibration status for selected target/device Bits are encoded as following b0 - No error was detected during re-calibration process b1 - Error was detected during re-calibration process

Description

Function gets ZQ calbration stat.

Return Value

0 on success

EINVAL if calStat is NULL

7.61. HPNFC_ClearZqStat

Function Declaration

`int32_t HPNFC_ClearZqStat(`*`privateData`*`, `*`calStatToClear`*`);`

`void *privateData`

`uint16_t calStatToClear`

Parameter List

`privateData` [in]

driver instance data filled by init

`int]` [ ]

calStatToClear Each bit of calStat points ZQ recalibration status to clear for selected target/device Bits are encoded as following b0 - do not clear error b1 - clear re-calibration error

Description

Function clears ZQ calbration stat.

Return Value

0 on success

EINVAL if calStatToClear is NULL

7.62. HPNFC_SetTimeoutConfig

Function Declaration

`int32_t HPNFC_SetTimeoutConfig(`*`privateData`*`, `*`timeoutConfig`*`);`

`void *privateData`

`HPNFC_TimeoutConfig *timeoutConfig`

Parameter List

`privateData` [in]

driver instance data filled by init

`in` [ ]

timeout configuration to set] timeoutConfig

Description

Function sets timeout configuration.

Return Value

0 on success

EINVAL if timeoutConfig is NULL

7.63. HPNFC_GetTimeoutConfig

Function Declaration

`int32_t HPNFC_GetTimeoutConfig(`*`privateData`*`, `*`timeoutConfig`*`);`

`void *privateData`

`HPNFC_TimeoutConfig *timeoutConfig`

Parameter List

`privateData` [in]

driver instance data filled by init

`out` [ ]

current timeout configuration] timeoutConfig

Description

Function gets timeout configuration.

Return Value

0 on success

EINVAL if timeoutConfig is NULL

7.64. HPNFC_SetCePinReduction

Function Declaration

`int32_t HPNFC_SetCePinReduction(`*`privateData`*`, `*`enable`*`);`

`void *privateData`

`uint8_t enable`

Parameter List

`privateData` [in]

driver instance data filled by init

`in` [ ]

configure CE pin reduction 0 - disable CE pin reduction 1 - enable CE pin reduction] enable

Description

Function sets CE pin reduction.

Return Value

0 on success

EINVAL if enable is bigger than 1

7.65. HPNFC_GetCePinReduction

Function Declaration

`int32_t HPNFC_GetCePinReduction(`*`privateData`*`, `*`enabled`*`);`

`void *privateData`

`uint8_t *enabled`

Parameter List

`privateData` [in]

driver instance data filled by init

`out` [ ]

conifgure CE pin reduction] enabled

Description

Function sets CE pin reduction.

Return Value

0 on success

EINVAL if privateData is NULL

7.66. HPNFC_GetDeviceInfo

Function Declaration

`int32_t HPNFC_GetDeviceInfo(`*`privateData`*`, `*`deviceInfo`*`);`

`void *privateData`

`HPNFC_DevInfo *deviceInfo`

Parameter List

`privateData` [in]

driver instance data filled by init

`deviceInfo` [out]

current device layout

Description

Function gets information about NAND flash device.

Return Value

0 on success

EINVAL if deviceInfo is NULL